

Group number: 65

- Module: 3
- Emrik Dunvald 020208-5759, ADS
- Elias Samantzis 000715-6631, ADS
- Gusamanel@student.gu.se
- Gusdunvem@student.gu.se
- We hereby declare that we have both actively participated in solving every exercise. All solutions are entirely our own work, without having taken part of other solutions.
- Elias hours spent: 20 hours
- Emrik hours spent: 20 hours

Reading and Reflection:

The paper discusses how ML systems are easy and cheap to implement and deploy, but maintaining such a system can prove to be quite expensive and difficult in some cases. ML systems suffer the same maintenance problems as traditional code but with additional ML-specific issues. According to the authors of the paper, this incurs technical debt that could be difficult to detect as it exists at the system level instead of in the code. As the hidden technical debt accumulates, erosion occurs in the boundaries of the code, having an undesirable impact on the system's ability to perform its tasks. Another discussed accumulator of technical debt in the paper is data dependencies. Since there are no tools to detect data dependencies, meaning it's easy to build large data dependencies that are later hard to untangle, accumulating technical debt. Feedback loops add to this problem, as models can reinforce biases in their own training data or create hidden dependencies between systems, making debugging more difficult. ML anti-patterns such as glue code, pipeline jungles, and dead experimental code paths further increase the debt by creating tangled dependencies and risks. Configuration management can also be a source of technical debt, as the mismanagement of time, resources, and production can lead to an increase in technical debt.

To reduce technical debt, the authors of the paper suggest better dependency tracking, automated monitoring, and a focus on long-term system stability over short-term gains.

An important takeaway from this paper is that ML systems accumulate hidden debt through either unseen or unexpected factors, which can lead to an unexpectedly high accumulation of hidden technical debt. An example of such an unexpected factor would be how data dependencies can be more problematic than code dependencies. Another important takeaway is that technical debt is not a strictly concrete term but more of a metaphor to communicate the issues that can stem from ML systems. More questions should be considered when assessing or addressing issues related to ML systems. Most importantly, the paper suggests further

development in this area of research to develop solutions for technical debt and other ML-related issues.

Discussion:

The system we developed did not have higher accuracy for the training data compared to the test data. Here we had around a 10% accuracy increase going from the first two cities, which we trained the model on, to the two other cities. There are a few reasons why this might happen, most likely is that the data of the two training cities is more noisy and has more outliers. The kmeans classifier can be quite sensitive to noise and outliers which means that if the training data have these qualities, it can be difficult to make accurate predictions on this data. We are assuming that the data for all 4 cities will be the same which most likely isn't the case and results in lower accuracy. It could be the case here that the two first cities have very different underlying data causing this set of data to be very difficult to predict. The cities we used for testing on the other hand might be much more similar and might have more defined clusters which makes it easier for the model to make predictions. What we ideally would want to do is to have one classifier for each city or train the classifier on data randomly sampled from all cities since that would mitigate some of the differences in the data. We might also want to remove outliers in the data since outliers can have a big impact on the model.

Another reason why the testing data might have a higher accuracy than the training data is that we are using a k which is more suitable for the second dataset. We didn't try that many different k for our model and it could be the case that we could improve the model by simply changing the number of assumed clusters. There are several methods for testing this but we also have to be careful not to overfit the model on the training data. With a very high k we might start to include random noise in the model, making the test accuracy worse which we don't want.

We ran the code a couple of times and saw that the accuracy for the test cities actually would vary quite a bit, somewhere between 65% and 85%. This would vary everytime we trained the model on the original data. The accuracy for the training data on the other hand would not vary and would stay at around 75%. It's hard to say why this is happening but we believe it has to have something to do with the underlying data distribution and that it might be very skewed. We looked at the cluster labels and saw that most of them were 0 which means that the accuracy of the prediction for the test cities is largely dependent on how many of them are 0.

Untitled

February 10, 2025

```
[17]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

beijing = pd.read_csv('Beijing_labeled.csv')
shenyang = pd.read_csv('Shenyang_labeled.csv')
combined = pd.concat([beijing, shenyang])
y = combined.loc[:, 'PM_HIGH']
X = combined.drop(columns='PM_HIGH')

print(combined)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳ random_state=3)
```

	season	DEWP	HUMI	PRES	TEMP	Iws	precipitation	cbwd_NE	\
0	4	-8.0	79.00	1026.0	-5.0	23.69	0.0	0	
1	4	-11.0	85.00	1021.0	-9.0	105.93	1.1	0	
2	4	-21.0	43.00	1030.0	-11.0	117.55	0.0	0	
3	4	-25.0	33.00	1034.0	-12.0	39.35	0.0	1	
4	4	-24.0	30.00	1034.0	-10.0	59.00	0.0	1	
..	...	...	...	...	...	...	...	...	
819	4	-22.0	50.63	1038.0	-14.0	4.00	0.0	0	
820	4	-16.0	52.47	1034.0	-8.0	4.00	0.0	0	
821	4	-13.0	45.91	1028.0	-3.0	40.00	0.0	0	
822	4	-4.0	80.04	1029.0	-1.0	6.00	0.0	0	
823	4	-8.0	79.42	1033.0	-5.0	4.00	0.0	0	

	cbwd_NW	cbwd_SE	PM_HIGH
0	0	1	1.0
1	0	1	0.0
2	1	0	0.0
3	0	0	0.0
4	0	0	0.0
..	...	...	...
819	0	0	1.0
820	0	0	1.0
821	0	1	1.0

822	1	0	1.0
823	0	0	1.0

[2895 rows x 11 columns]

```
[24]: import random

class Kmeans:
    def __init__(self, k):
        self.k = k
        self.clusters = []
        self.cluster_labels = []
        self.cluster_centers = []

    def fit(self, X, y):
        targets = y.to_numpy()
        features = X.to_numpy()
        best_sse = np.inf # Remember the best cluster
        max_iter = 50 # Maximum number of iterations
        random_inits = 10 # Number of random initializations

        for _ in range(random_inits): # Try different random starts for better
↪result
            cluster_centers = [features[random.randint(0, len(features) - 1)]
↪for _ in range(self.k)]
            clusterwise_sse = 0

            for _ in range(max_iter): # Iterate untill "convergence"
                clusters = [[] for _ in range(self.k)]
                cluster_labels = [[] for _ in range(self.k)]

                for target, feature in zip(targets, features): # Iterate over
↪each datapoint
                    dists = [np.linalg.norm(feature - center) for center in
↪cluster_centers] # Get all cluster distances to this datapoint
                    min_dist = np.min(dists) # Find the closest cluster
                    index = dists.index(min_dist) # Find the index of the
↪closest cluster

                    # Add the datapoint to that cluster
                    clusters[index].append(feature)
                    cluster_labels[index].append(target)

            for n in range(len(cluster_centers)):
                cluster_centers[n] = (sum(clusters[n]) / len(clusters[n]))
↪# Compute the cluster centers
```

```

        for centroid, points in zip(cluster_centers, clusters):
            sse = sum([np.square(centroid - point) for point in points]) #
→ Compute sse
            clusterwise_sse += sum(sse) # Sum the sse

        if clusterwise_sse < best_sse: # Update the clusters if we get a
→ better sse on this init
            best_sse = clusterwise_sse
            self.clusters = clusters
            self.cluster_labels = cluster_labels
            self.cluster_centers = cluster_centers

            sums = [sum(self.cluster_labels[i]) / len(self.
→ cluster_labels[i]) for i in range(len(self.cluster_labels))] # The average
→ of all labels in the cluster
            rounded = round(sums[0], 0) # round it to 1 or 0
            self.cluster_labels = [round(i, 0) for i in sums] # Set the
→ cluster labels

    def predict(self, X):

        predictions = []
        features = X.to_numpy()

        for feature in features:
            dists = [np.linalg.norm(feature - self.cluster_centers[i]) for i in
→ range(len(self.cluster_centers))] # compute distances to all clusters
            min_dist = np.min(dists) # Find the closest cluster
            index = dists.index(min_dist)
            predictions.append(self.cluster_labels[index])

        return predictions

    def accuracy(self, y_pred, y_true):
        pred = np.asarray(y_pred)
        true = np.asarray(y_true)
        accuracy = np.average(np.equal(pred, true))

        return(accuracy)

```

```

[25]: kms = Kmeans(8)
      kms.fit(X_train, y_train)
      prediction = kms.predict(X_test)

```

```
ac = kms.accuracy(prediction, y_test)

print(ac)
```

0.7737478411053541

```
[26]: guangzhou = pd.read_csv('Guangzhou_labeled.csv')
      shanghai = pd.read_csv('Shanghai_labeled.csv')

      test = pd.concat([guangzhou, shanghai])

      y = test.loc[:, 'PM_HIGH']
      X = test.drop(columns='PM_HIGH')

      prediction = kms.predict(X)
      ac = kms.accuracy(prediction, y)

      print(ac)
```

0.8701442841287459

0.0.1 Implementation

The code implements a rather simple and unoptimized kmeans classifier. The class itself has the number of clusters (k), the clusters themselves, the centroids and the cluster labels as its variables. It initializes a random start 10 times (this was chosen arbitrarily) and lets the centroids converge over 50 loops. After 50 loops of updating it considers convergence and calculates the sum of squared errors. If the new sse is lower than that of the previous best sse, then we update all the models variables, otherwise we don't use the results from this random initialization.

0.0.2 Apply your classifier

There are some interesting results to discuss here. We see that when we evaluate the model on the two first cities Beijing and Shenyang, we get a pretty underwhelming accuracy around 75%. This accuracy could likely be improved by tweeking a few of the hyperparameters such as the number of random inits, k and evaluation metric (silhouette score instead of sse for example). Whats a bit surprising is the fact that we get a higher accuracy for the last two cities Guangzhou and Shanghai, around 85%.

[]: